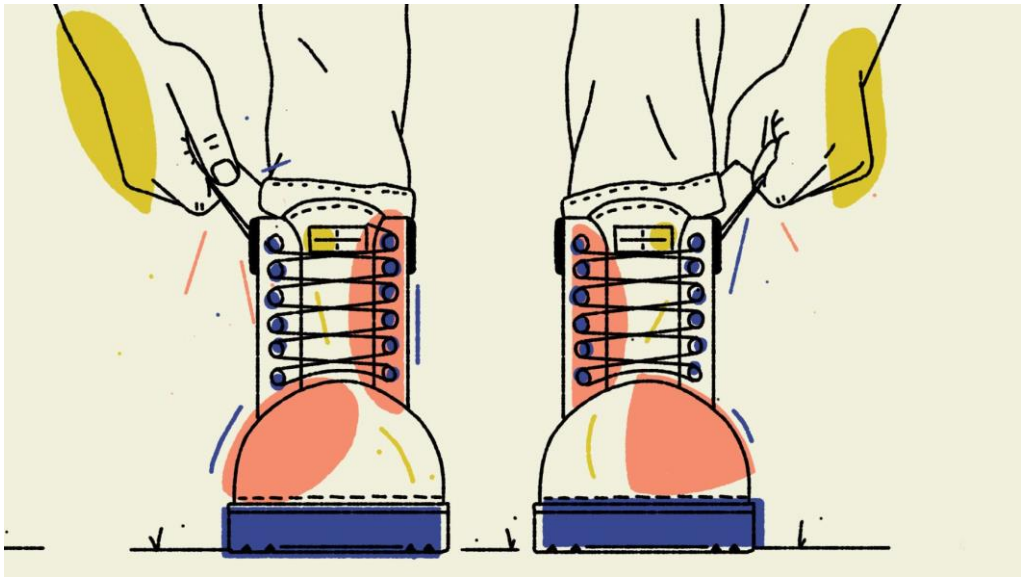
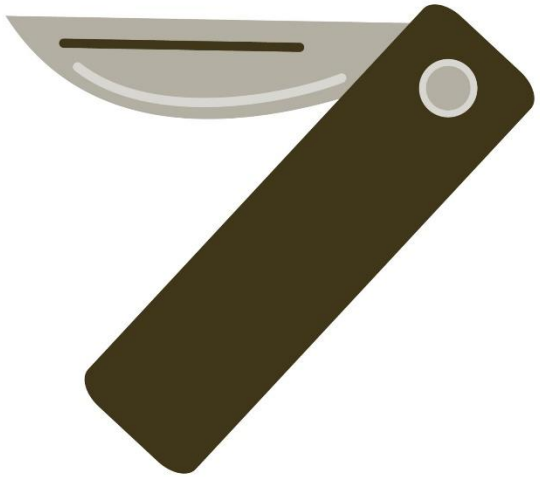
Lecture 8 – Resampling Techniques





Today's Learning Outcomes

1. Be able to describe the differences between bootstraps, jackknifes, and permutations approaches to resampling

2. Be able to build bootstrap, jackknife, and permutation functions in Python

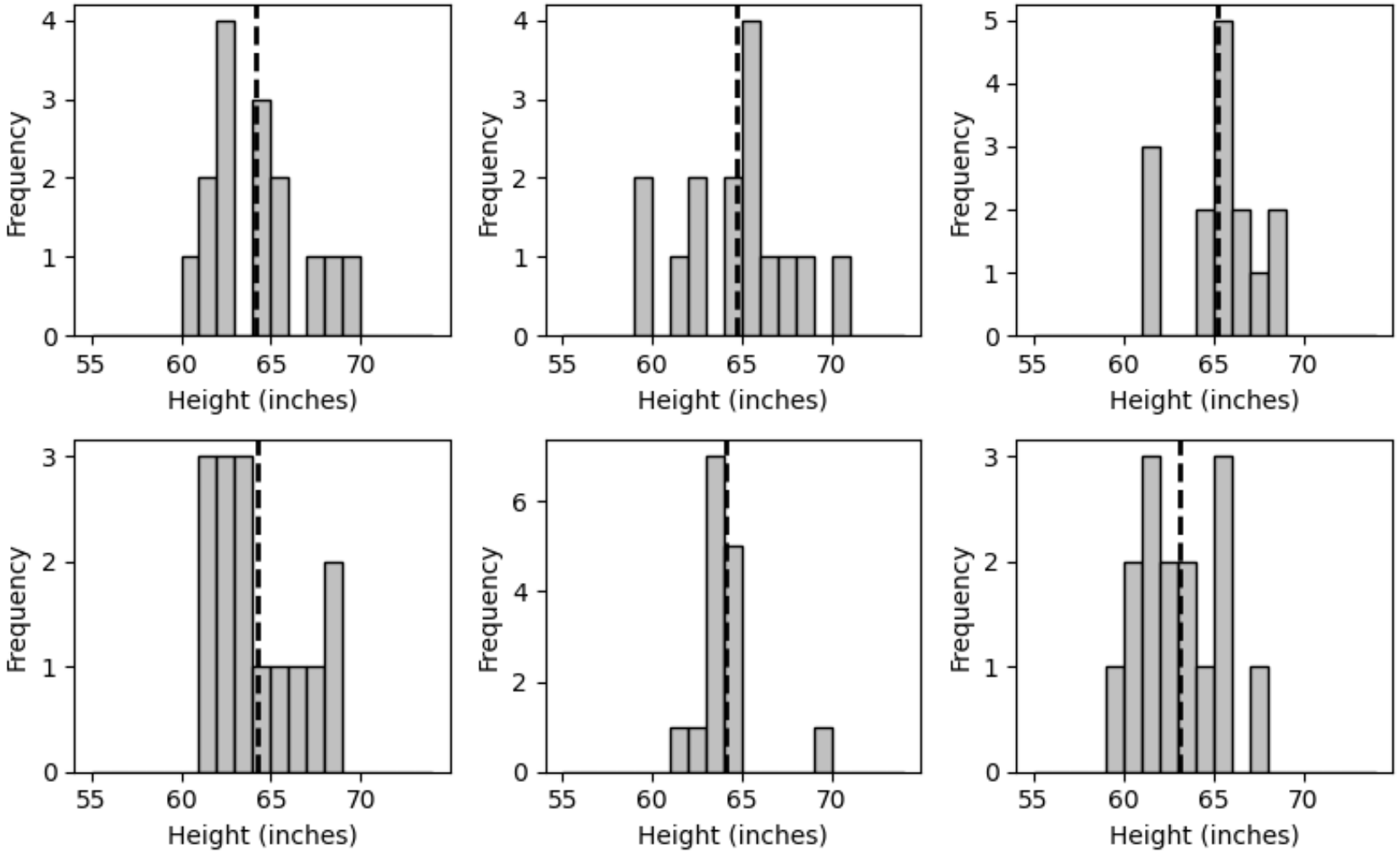
Measuring Uncertainty

- We talked about measuring variation of a dataset
  - Percentiles, standard deviation, standard error, range

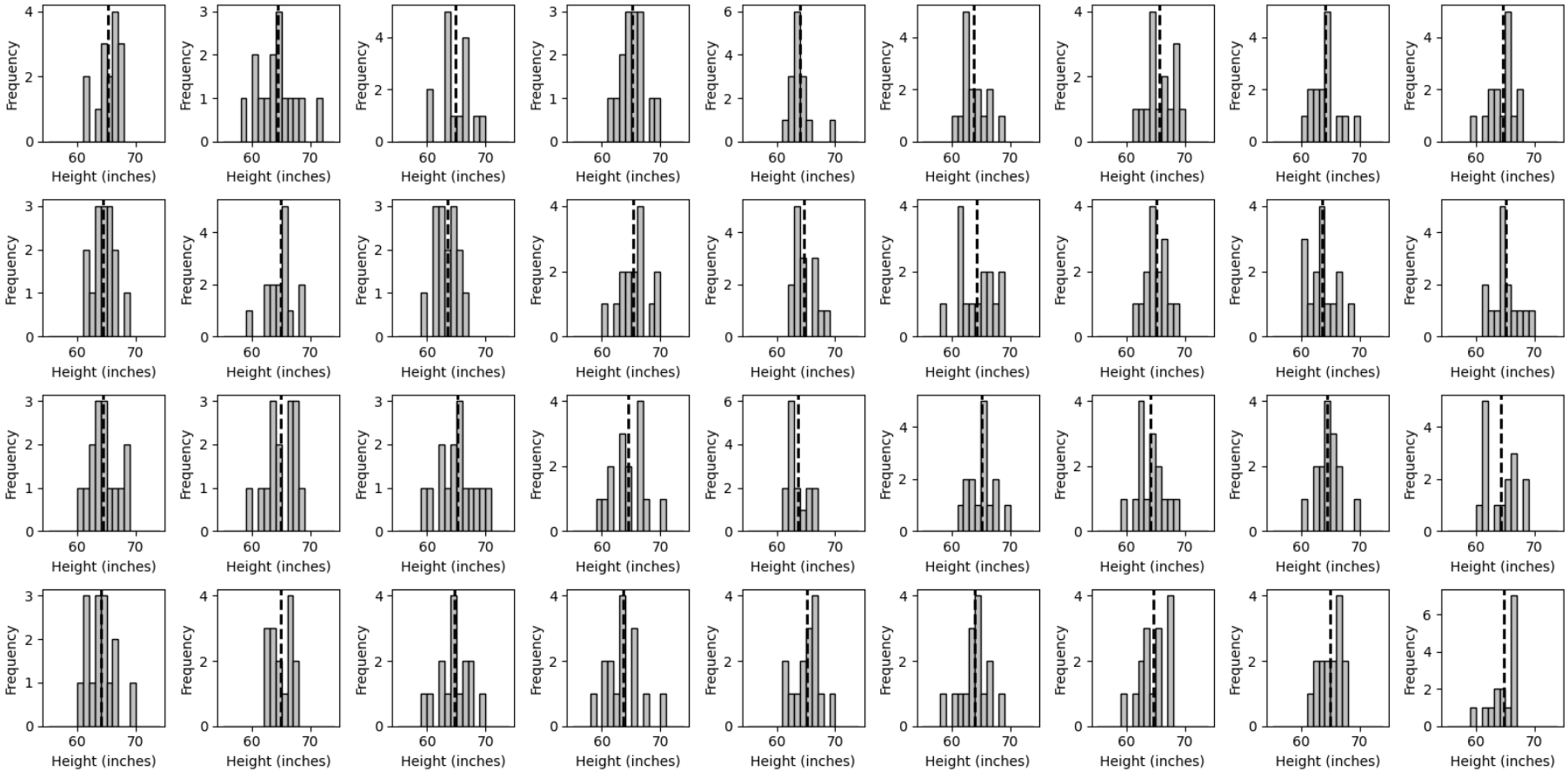
- But we often want to measure variation in our estimated value from a sample
  - Ex. we calculate a mean height from everyone present in the class as a height estimate of the population of the earth sciences department members
  - How variable is that mean estimate if I took a different sample?

- This variation in an estimated value is our uncertainty

Six samples of heights (15 individuals pulled from a normal distribution) from the same population showing uncertainty in the mean



Measuring Uncertainty



What if we repeat it more times?

Measuring Uncertainty

- After taking many samples can then calculate the standard deviation, standard error, percentiles of the distribution of mean values to get
  a measure of uncertainty in the sample mean

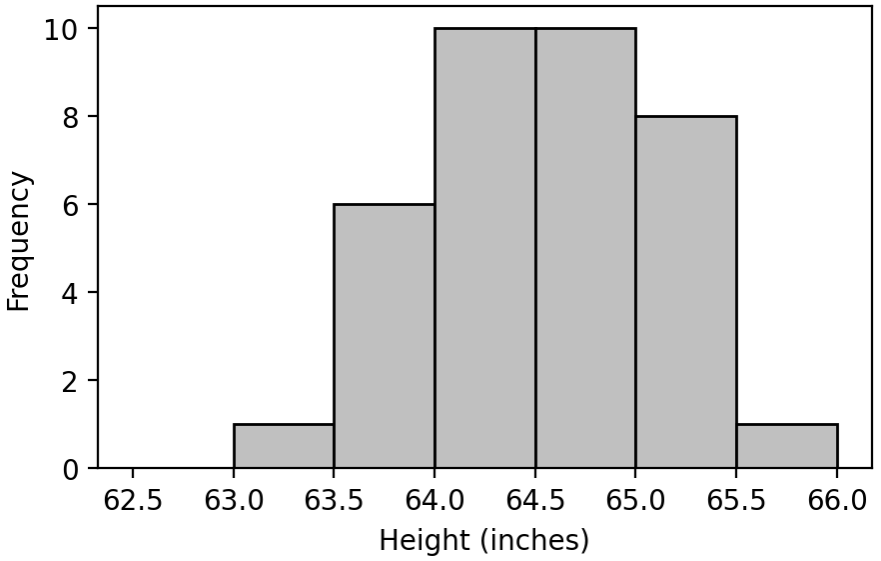
mean(36 samples) = 64.53

sd(36 sample means) = 0.56

95% CI of mean estimate =
63.54 – 65.39

True mean = 64.5

Means of samples (n = 36)



Limits on Measuring Uncertainty

- Ideally, we would take many samples from a population to estimate uncertainty in the statistic of interest
  - I might be able to measure every person in the Earth Sciences department, but no way I could do all of UB
  - Sometimes impossible to sample more
    - Too costly
    - Cannot return to a location
    - Population of interest no longer exists (rocks destroyed, extinct organisms)

Resampling

- So instead we can resample the data we do have to estimate uncertainty

- Bootstrap ← Mimics data ($n_{bootstrap} = n_{data}$)
- Jackknife ← Degrades data ($n_{jackknife} < n_{data}$)
- Permutation ← Scrambles data ($n_{permutation} = n_{data}$)

All of these are general approaches, rather than specific formulas or tests

Bootstrap

Given a dataset of length *n* a bootstrap…

1. Resamples values from that dataset at random, **with replacement**, until it has *n* values

2. Calculate the metric(s) of interest from this new sample

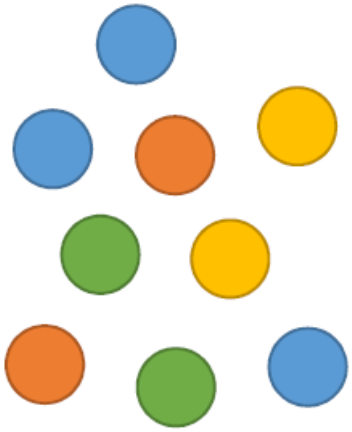
3. Repeat step 1 and 2 many times to get a set of your metric(s)

4. Calculate the standard deviation, standard error, or percentile ranges to measure the metric(s) uncertainty

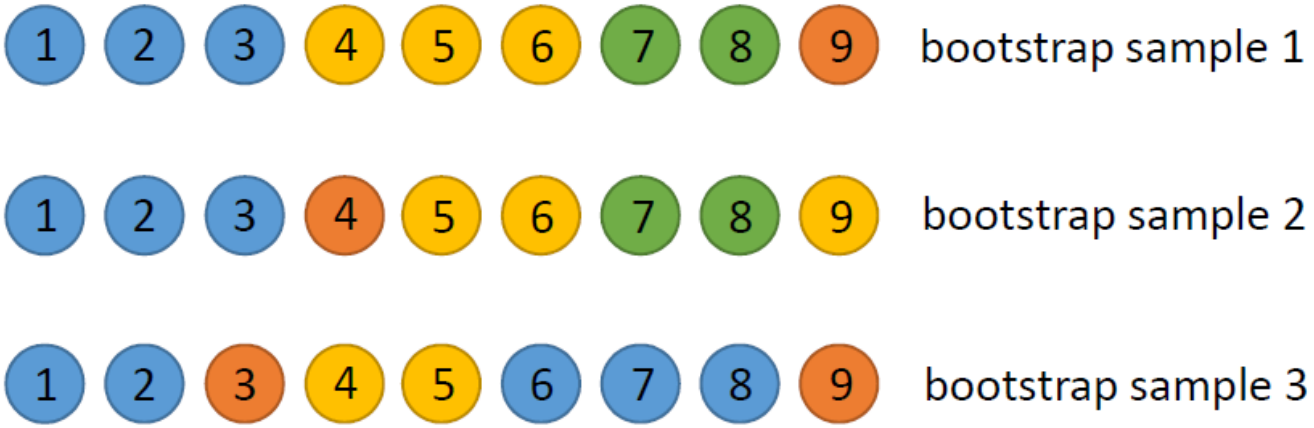
Bootstrap

Can get repeat values

Can entirely miss values



original sample



- Extremely generalizable
  - In the above example our metric could be the proportion or number of blue balls

- Bootstrap assumes data are random samples and independent, but does **not** assume normality of the underlying data

Bootstrap Example Code

```
# Generate 50 random heights from N(mean=64.5, sd=2.5)
heights = stats.norm.rvs(loc=64.5, scale=2.5, size=50)


# SciPy's bootstrap requires data as a tuple of arrays
result = bootstrap((heights,), np.median, n_resamples=100)
```

Documentation for bootstrap function in Python: https://docs.scipy.org/doc/scipy-1.17.0/reference/generated/scipy.stats.bootstrap.html

*Both gave the same answer here, not a guarantee!

Bootstrap Example Code

```
# SciPy's bootstrap requires data as a tuple of arrays
result = bootstrap((heights,), np.median, n_resamples=200)
print("Bootstrap median estimate:", result.bootstrap_distribution.mean())
print("95% CI:", result.confidence_interval)
```

Output:

```
Bootstrap median estimate: 64.87073456677295
95% CI: ConfidenceInterval(low=np.float64(64.07774055038485), high=np.float64(65.60286883123302))
```

When to Bootstrap

Bootstrapping can be used…

1. in cases where there is no available analytic formula for the standard error (ex. the median)

2. when the data (and resulting metric(s) distribution) are strongly skewed
   - i.e. when normality is violated

3. when additional samples are difficult/impossible to obtain

Issues with bootstraps

- When the sample size is small, under 100 or so, bootstrap methods have difficulties producing high quality estimates of confidence intervals, particularly high (over 95%) or low (under 5%)
  - This makes some real sense, it is hard to talk about 1 % chances if you only have 100 specimens

- Assumes that your original sample is representative of the population of interest
  - Known or unknown biases of the data will propagate

Parametric Bootstraps

- Can try to work around limits by incorporating some other types of information into the bootstrap procedure

- If we can identify that the data follows a specific distribution, we can make use of that distribution in the bootstrap procedure, making the process a *parametric bootstrap procedure*
  - Better estimates of edge cases (if correct)

Parametric Bootstraps

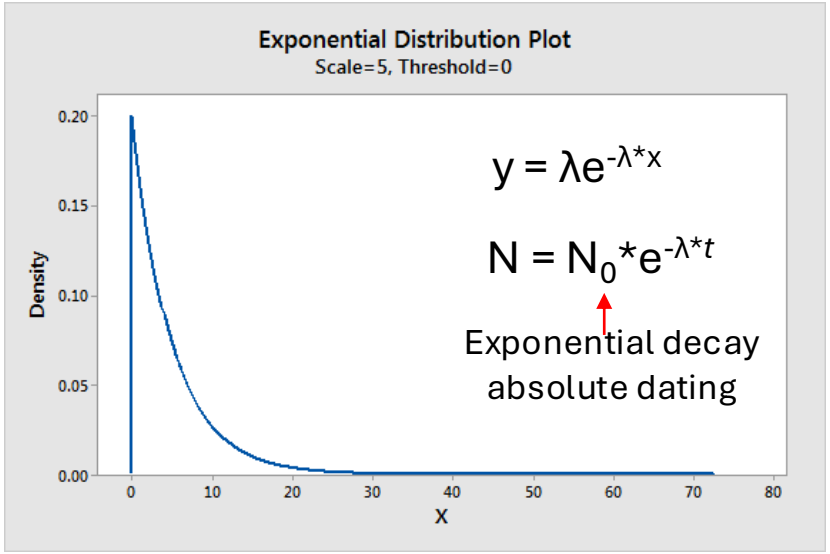
- Distribution models have parameters in them
  - Ex. mean and standard deviation for a normal distribution

- In a parametric bootstrap…
  1. resample the data with replacement
  2. estimate the parameters from the bootstrap set
  3. feed the parameter values into the distribution model
  4. use the distribution to calculate the statistic of interest, and store it
  5. repeat steps 1-4 many times, report the confidence interval over the parametric bootstrap

Parametric Bootstraps

Example: Species durations

If we have a set of 15 species durations, and we *assume* that species faced a constant extinction risk, then species durations will follow an exponential distribution

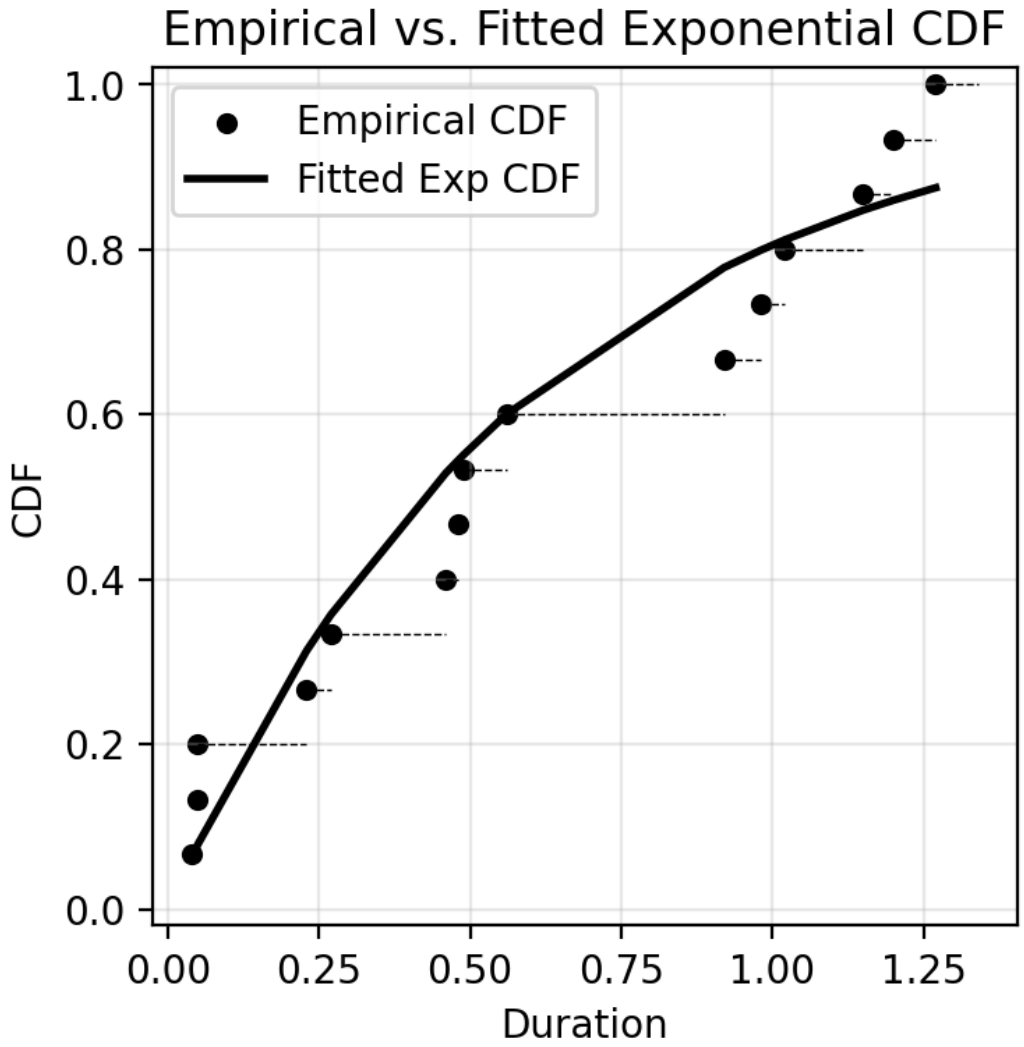


Based on our fifteen data points, we would like to know the 90% upper bound of species durations, and the uncertainty in that estimate.

Here is our set of data points, in millions of years

1.02, 1.20, 0.49, 0.27, 0.05, 1.15, 0.92, 0.98, 0.04, 0.23, 0.46, 0.56, 0.05, 0.48, 1.27

Here is the cumulative distribution function for the data, and the fitted exponential cumulative distribution



For an exponential function, the mean value is equal to 1/(decay rate **λ)**

$N = N_0 * \exp(-\lambda * t)$

where N_0 is the initial number of species

The mean here was 0.611, so the decay rate is 1.636

We can get the mean from Python, and then use the quantile calculation function for the exponential distribution

```
stats.expon.ppf(0.9,scale=0.611)
[1]1.40757 = 1.41
```

So our estimate, based on the exponential distribution of the 90^{th} percentile is 1.41

What is the confidence interval on this?

Use the parametric form of the bootstrap

Parametric Bootstrap Example Code

```
def exp_quart_par_boot(x, nboots, random_state=None):
    rng = np.random.default_rng(random_state)
    x = np.asarray(x)
    n = x.size
    qval = np.empty(nboots, dtype=float)
    for i in range(nboots):
        pbx = rng.choice(x, size=n, replace=True)
        pbmean = pbx.mean() # exponential scale = mean
        qval[i] = stats.expon.ppf(0.90, scale=pbmean)
    return np.std(qval, ddof=1),qval
```

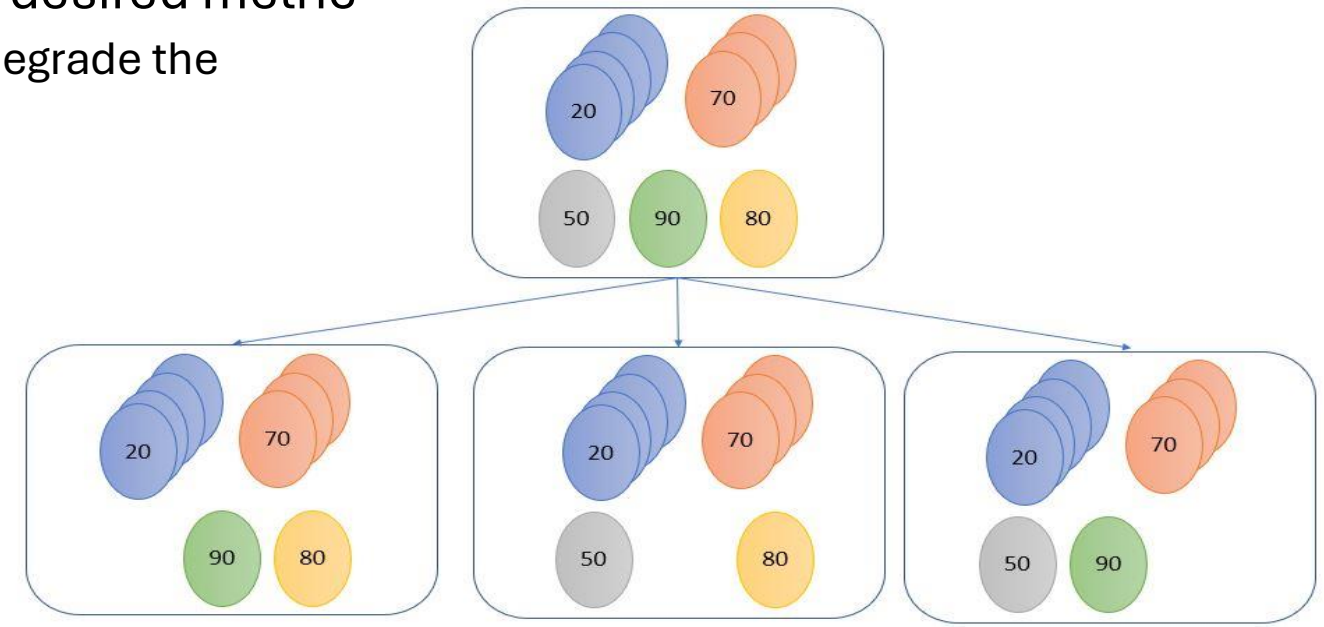
The bootstrap mean is used in each step as the input to the exponential function model, which is then used to compute the 90th percentile of values, based on the mean

- Any gain in accuracy is dependent on the chosen model being correct

Jackknife

- A method of subsampling data to check the sensitivity or robustness of an effect

- Process is to drop a single data point, category, or proportion of the data and then recalculate the desired metric
  - Purposefully degrade the actual signal



Jackknife Example Code

```
def jackknife_durs_one_by_one(x):
   x = np.asarray(x, dtype=float)
   n = x.size
   qval = np.empty(n)
   for i in range(n):
      pbx = np.delete(x, i) # leave-one-out sample
      pbmean = pbx.mean() # exponential scale = mean
      qval[i] = expon.ppf(0.90, scale=pbmean)
   y = (qval.min(), qval.max())
   full_val = expon.ppf(0.90, scale=x.mean())
   print(f"Full dataset value = {full_val}")
   return y
```

When to Jackknife

- When there's concern that a few data points might be responsible for a signal
  - Unequal distribution of data sources or groups of data
- When you want to know how sensitive a result is to changes in the dataset
  - More sensitive measures tend to have higher uncertainty

Permutation

- A method of resampling to test a hypothesis (rather than the uncertainty in a method)
  - Permute the data and recalculate the effect size (% variance explained, p-value, etc.)

- Works by scrambling associations in the real data to measure the effect size created at random

Example

A company wants to know if a new training program actually improves employee productivity.

They run a small pilot test:

- Group A (Training): 8 employees
- Group B (No Training): 8 employees

After 1 month, they measure productivity scores. Suppose the data is:

```
group_training = [92, 95, 88, 90, 93, 96, 91, 94]
group_control  = [86, 82, 85, 84, 83, 87, 80, 88]
```

The training group looks better — but is that difference real or could it be due to random chance?

Example Code

We randomly shuffle the labels, pretending the training assignment didn't matter, and compute the difference each time.
This creates a null distribution — what differences you'd expect by chance alone.

```
obs_diff = group_training.mean() - group_control.mean()
print("Observed difference:", obs_diff)
combined = np.concatenate([group_training,
group_control])
nA = len(group_training)
perm_diffs = []
for i in range(10000):
   shuffled = np.random.permutation(combined)
   permA = shuffled[:nA]
   permB = shuffled[nA:]
   perm_diffs[i] = permA.mean() - permB.mean()
p_value = np.mean(perm_diffs >= obs_diff)
print("Permutation p-value:", p_value)
```

When to Permute

- Permutations are most useful when you are looking at effect sizes (i.e. signal strength) in a dataset

- Need to know how strong an effect can be by random chance?
  - Permutation is a good approach

Summary Resampling

- All these techniques are random resampling and thus if you run the same technique over and over you will get different results
  - As the number of resamplings increases your variance in uncertainty will decrease (with diminishing returns)

- Bootstrap <- measures uncertainty in a metric
- Jackknife <- measure sensitivity of a metric
- Permutation <- measures strength of a null signal

Today's Learning Outcomes

1. Be able to describe the differences between bootstraps, jackknifes, and permutations approaches to resampling

2. Be able to build bootstrap, jackknife, and permutation functions in Python